# SSH Brute-Force Detection, Alerting, and Firewall Response Lab

*Linux log analysis • Bash • Cron • UFW • Packet inspection*

*A practical Linux security-monitoring lab using Bash, cron, systemd, UFW, and tcpdump*

## 1. Executive Summary

This lab presents a lightweight approach to identifying repeated SSH password-authentication failures. A Bash detector reads recent SSH journal entries, filters failed-password messages, groups matching events by source IP address, and compares each count with a configured threshold. When the threshold is met, the script records an alert and broadcasts a warning to the logged-in terminal. The report describes a learning-lab implementation and the observed tests; it does not claim production-grade detection accuracy.

The lab used Kali Linux and Ubuntu Desktop as SSH clients, with Ubuntu Server as the target. Testing used TCP port 2222. Packet capture showed traffic from 192.168.229.130 to 192.168.229.128 on TCP port 2222. The multiple-source detection test separately recorded alerts for 192.168.229.130 and 192.168.229.131. During firewall troubleshooting, the UFW rules initially listed a broad allow rule before the source-specific reject rule. After I inserted the reject rule at position 1, a subsequent connection attempt returned 'Connection refused'. This result was consistent with the revised firewall configuration, but the test did not establish rule order as the sole cause of the earlier behavior.

This is a learning lab, not a production SIEM. Detection relies on cron-based polling, so alerts may be delayed. The firewall rule was applied manually; the detector does not automatically block the source. The documented results show the behavior observed in the tested scenarios, while threshold boundary cases, duplicate-alert handling, IPv6 behavior, and other scenarios remain to be validated.

## 2. Objective and Detection Rule

The objective is to detect repeated failed SSH password-authentication events from the same source IP address and notify an administrator for investigation.

**The configured detection rule is intended to generate a "Possible SSH Brute-Force Activity" alert when one source IP address has at least five failed SSH password-authentication events within the preceding three-minute lookback window.**

The detector is scheduled by cron to run once per minute and checks a rolling three-minute lookback window using journalctl to read the systemd journal. Because successive windows overlap, an event may be counted in more than one run and may contribute to repeated alerts. The documented multiple-source test produced alerts for six failed-password events per source; separate controlled tests at, below, and above the threshold are still needed to validate boundary behavior.

## 3. Lab Environment and Architecture

The lab was conducted in VMware using three virtual machines:

| Machine | Role | Address / details |
|---|---|---|

| | | |
|---|---|---|
| Kali Linux | SSH client / simulated attacker | 192.168.229.130 |
| Ubuntu Server | SSH target, logs, Bash detector, UFW | 192.168.229.128; SSH TCP/2222 |
| Ubuntu Desktop | Additional lab host used in testing | 192.168.229.131 |

### Logical detection and response flow

Kali Linux / Ubuntu Desktop — SSH test attempts → Ubuntu Server (TCP/2222)

Ubuntu SSH journal → journalctl → Bash filtering and source-IP counting

Threshold reached → alert appended to a log and broadcast to logged-in terminal

Administrator reviews the alert and, if appropriate, applies a UFW rule manually

## 4. Initial Detection Design

The initial log review used journalctl to retrieve recent SSH service entries and grep to isolate messages containing "Failed password." This is a preliminary log-inspection step that focuses on password-authentication failures rather than all SSH activity. It does not, by itself, validate the script's source-IP extraction, counting, threshold evaluation, or notification behavior.

```
journalctl --since "3 minutes ago" --unit ssh --no-pager | grep "Failed password"
```

The detector was scheduled to run once per minute. The cron entry recorded in the lab notes is shown below:

```
$crontab -l
* * * * * /usr/local/bin/alert.sh
```

## 5. Simulation and Log Analysis

Repeated SSH login attempts were made from the client virtual machines to Ubuntu Server on TCP port 2222. The target journal was then inspected for failed-password events. The detector is intended to extract the source IP address from each matching message, count events by IP, and compare each count with the configured threshold. The detector implementation is presented in Section 6.1. The following evidence documents the SSH activity and test results observed during the lab.

A connection attempt does not necessarily represent a failed authentication. SSH logs also contain connection-closure and other messages. The detector is intended to count messages containing "Failed password," not every SSH log entry.

```
sudo journalctl --since "3 minutes ago" --unit "ssh" --no-pager | grep "Failed password"
Failed password for tadi from 192.168.229.130 port 41001 ssh2
Failed password for tadi from 192.168.229.130 port 41002 ssh2
Connection closed by authenticating user tadi 192.168.229.130 port 41002 [preauth]
```

The "Connection closed by authenticating user ..." message is an example of a different SSH event; it is not itself a failed-password event and should not be counted as one by a detector that filters only "Failed password" messages.

## 6. Detection Logic and Automated Alerting

1. Read recent entries from the SSH service journal.
2. Filter for failed password-authentication messages.
3. Extract the source IP address from each matching event.
4. Group and count events by source IP.
5. If an IP address meets or exceeds the threshold, create an alert containing the source IP, event count, lookback window, and timestamp.
6. Append the alert to a log and broadcast it to the logged-in terminal.

### 6.1 Bash Detection Script

The following Bash script implements the detection logic described above. It reads recent SSH journal entries, extracts source IP addresses from failed-password messages, counts events by source IP, and generates an alert when an address reaches the configured threshold of five events within the three-minute lookback window. Alerts are broadcast using wall and appended to /var/log/alert.log. In the documented test, the alert was observed in the logged-in terminal on Ubuntu Server.

```
#!/bin/bash
SIREN=$(printf '\U0001f6a8 CRITICAL SECURITY ALERT')
RESULT=$(journalctl --since "3 minutes ago" --unit "ssh"  \
| grep "Failed password"  \
| awk '{
        for (i=1; i<NF; i++){
                if ($i == "from") {
                        print $(i+1)
                        break
                        }
                }
        }' \
| sort \
| uniq -c \
| awk '$1 >= 5 {print $1, $2}')
#NF=Number of Fields
if  [[ -n "$RESULT" ]]; then
while read -r ATTEMPT IP; do
MESSAGE="$SIREN: Possible SSH Brute Force
Source IP: $IP
Number of Attempt: $ATTEMPT
Time block: 3 minutes
$(date)"
printf '%s\n' "$MESSAGE" | wall
```

```
printf '%s\n' "$MESSAGE" >> /var/log/alert.log
done  <<< "$RESULT"
fi
```

### Detection of Multiple Source IP Addresses

In a test involving two source IP addresses, the script generated separate alerts for 192.168.229.130 and 192.168.229.131, reporting six failed-password events for each address within the configured three-minute lookback window. This demonstrates that the script reported the two sources independently in that test. It does not establish performance under all traffic patterns or threshold boundary conditions.

Failed SSH authentication
↓
Extract source IP
↓
Group by source IP
↓
Count attempts
↓
If count >= 5 within 3 minutes
↓
Generate alert

Cron runs every minute and checks SSH journal entries from the preceding three minutes. This is scheduled polling, not continuous, event-driven monitoring. Since successive lookback windows overlap, the same events can trigger repeated alerts. Repeated alerts were observed during testing; the script does not currently track previously alerted events or deduplicate notifications.

**Cron entry recorded in the lab notes:**

```
$crontab -l
* * * * * /usr/local/bin/alert.sh
```

SECURITY ALERT

Possible SSH Brute-Force Activity

Source IP: [source IP]

Failed-password events: [count]

Lookback window: 3 minutes

Detected: [timestamp]

## 7. Firewall Response

The intended response was to reject TCP connections from the observed source IP address to port 2222. This rule was applied manually on Ubuntu Server; the Bash detector did not apply the firewall rule automatically.

Accordingly, the implemented workflow provides detection and notification, while firewall enforcement remains a separate administrator action.

```
sudo ufw insert 1 reject from 192.168.229.130 to any port 2222 proto tcp
```

The numbered UFW output showed a broad allow rule for TCP/2222 listed before the source-specific reject rule. I inserted the reject rule at position 1, ahead of the broad allow rule, and verified the resulting order. In the subsequent test, Kali reported "Connection refused." This observation was consistent with the revised firewall configuration rejecting the connection, but it does not prove that rule order was the sole cause of the earlier behavior.

```
sudo ufw status numbered
[1] 2222/tcp REJECT IN 192.168.229.130
[2] 2222/tcp ALLOW IN Anywhere
[3] 22/tcp ALLOW IN Anywhere
```

# 8. Troubleshooting and Investigation

## 8.1 SSH service and port 2222

After installing OpenSSH Server, I found that the SSH service was initially inactive. I started the service using the following command:

```
sudo systemctl start ssh
```

I then configured SSH to listen on TCP port 2222 by editing `/etc/ssh/sshd_config` and adding the following directive as shown in Figure 6:

```
Port 2222
```

After saving the configuration, I restarted the SSH service to apply the change:

```
sudo systemctl restart ssh
```

This configuration allowed the target Ubuntu Server to accept SSH connection attempts on TCP port 2222, which was used for the subsequent detection and firewall-response tests.

```
ss -ltpn | grep 2222
```

The command returned the following output:

```
LISTEN 0 4096    0.0.0.0:2222      0.0.0.0:*
LISTEN 0 4096    [::]:2222         [::]:*
```

The output showed listeners on TCP port 2222 bound to the wildcard IPv4 address (0.0.0.0) and wildcard IPv6 address (::). This confirms the reported listening sockets; it does not by itself confirm end-to-end reachability from every client or prove that IPv6 connectivity and firewall filtering were fully tested.

I then configured UFW to permit incoming traffic on port 2222:

```
sudo ufw allow 2222
```

I checked the firewall status with:

```
sudo ufw status
```

Status: active

```
To                         Action    From
--                         ------    ----
2222                       ALLOW     Anywhere
22/tcp                     ALLOW     Anywhere
2222 (v6)                  ALLOW     Anywhere (v6)
22/tcp (v6)                ALLOW     Anywhere (v6)
```

These results showed that SSH was listening on port 2222 on IPv4 and IPv6 wildcard addresses and that UFW displayed allow rules for port 2222. These checks alone do not establish that connections from every source will succeed; effective connectivity also depends on the active firewall rules, routing, and client-side conditions.

### 8.2 APT/dpkg lock held by unattended-upgrade

During OpenSSH Server installation, APT reported that /var/lib/dpkg/lock-frontend was held by process 2863 (unattended-upgr). This indicated that another package-management process was using dpkg. Rather than forcibly terminating the active process, I waited for it to complete and release the lock. Installation then succeeded.

The package-manager lock helps prevent concurrent package-management operations from modifying package state simultaneously. Waiting for the active process to finish was therefore the safer approach in this situation.

### 8.3 UFW rule syntax

An early UFW command used incorrect syntax and returned a “too many arguments” message. The corrected source-specific rule was:

```
sudo ufw reject from 192.168.229.130 to any port 2222 proto tcp
```

In this command, from identifies the source IP address, to any leaves the destination address unrestricted, port 2222 specifies the destination port, and proto tcp limits the rule to TCP traffic.

### 8.4 Source-IP verification and NAT hypothesis

When the target remained reachable despite the intended firewall rule, I checked the interface addresses and SSH logs. I hypothesized that Network Address Translation (NAT) might be changing the source IP address observed by the target. To investigate this possibility, I used tcpdump to inspect incoming traffic on TCP port 2222:

```
sudo tcpdump -ni any tcp port 2222
```

The packet capture showed traffic from 192.168.229.130 to 192.168.229.128 on TCP port 2222. This supported the conclusion that the target observed the expected source IP address during this test. Therefore, the capture did not support the hypothesis that NAT was changing the observed source address in this instance. However, this result alone did not establish why the target remained reachable.

### 8.5 SSH key hypothesis

I considered whether a saved RSA public key could explain the observed behavior. SSH public-key authentication does not bypass a firewall rule that prevents the connection from reaching the SSH service; network filtering occurs before SSH authentication can complete. The available evidence did not indicate that public-key authentication explained the connectivity issue. This was a troubleshooting hypothesis, not a separately validated authentication test.

### 8.6 UFW rule ordering

The numbered UFW output showed the broad allow rule before the source-specific reject rule. I inserted the reject rule at position 1 and verified the new ordering. A subsequent SSH connection attempt from Kali returned "Connection refused." This was consistent with the revised firewall configuration, but the test did not independently isolate rule order as the cause of the earlier connectivity behavior.

## 9. Evidence and Figures

The report includes screenshots documenting client tests, target-side log detection, the detector, firewall behavior, packet capture, alert broadcasts, rule ordering, and the final connection test. Captions explain the purpose of each figure. The 16 numbered figures document the displayed test states; they do not establish results for scenarios that were not tested.

Figure 1. First SSH test launched from Kali Linux.

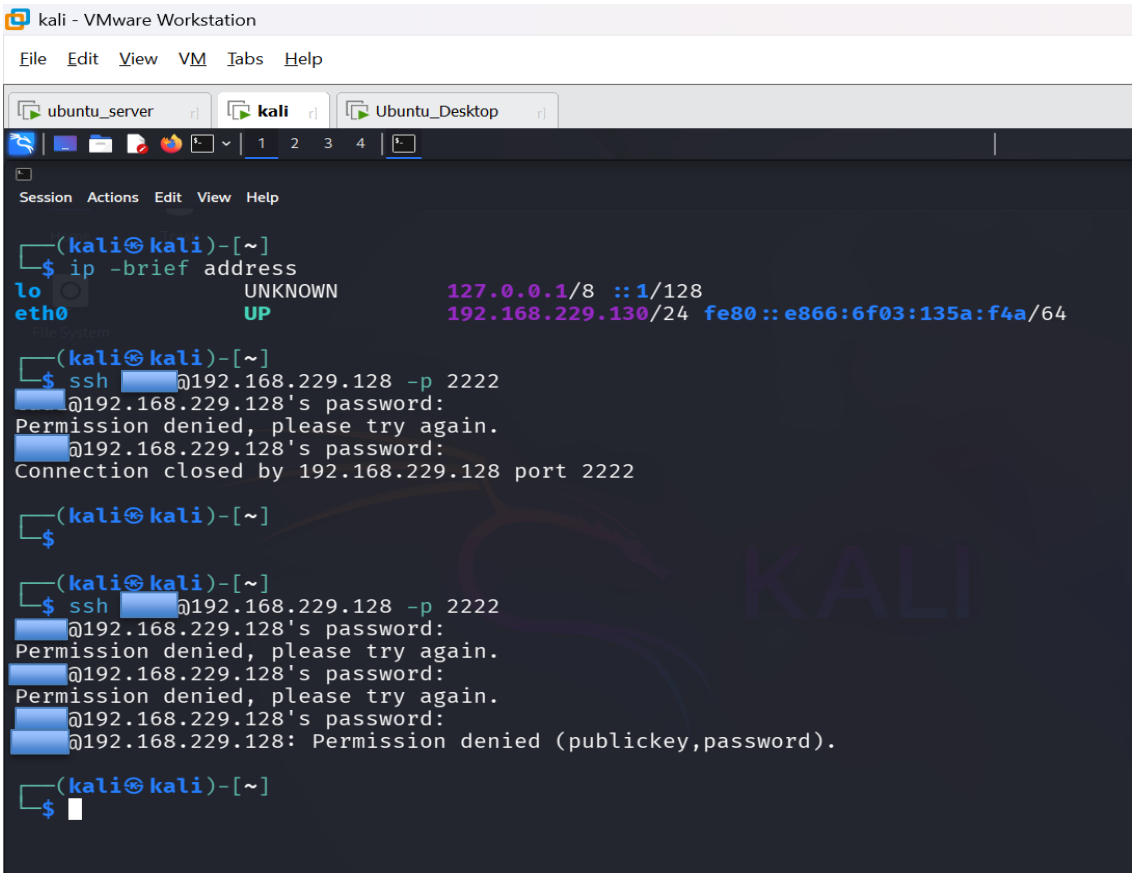



Figure 2. Ubuntu Server records the first test activity.

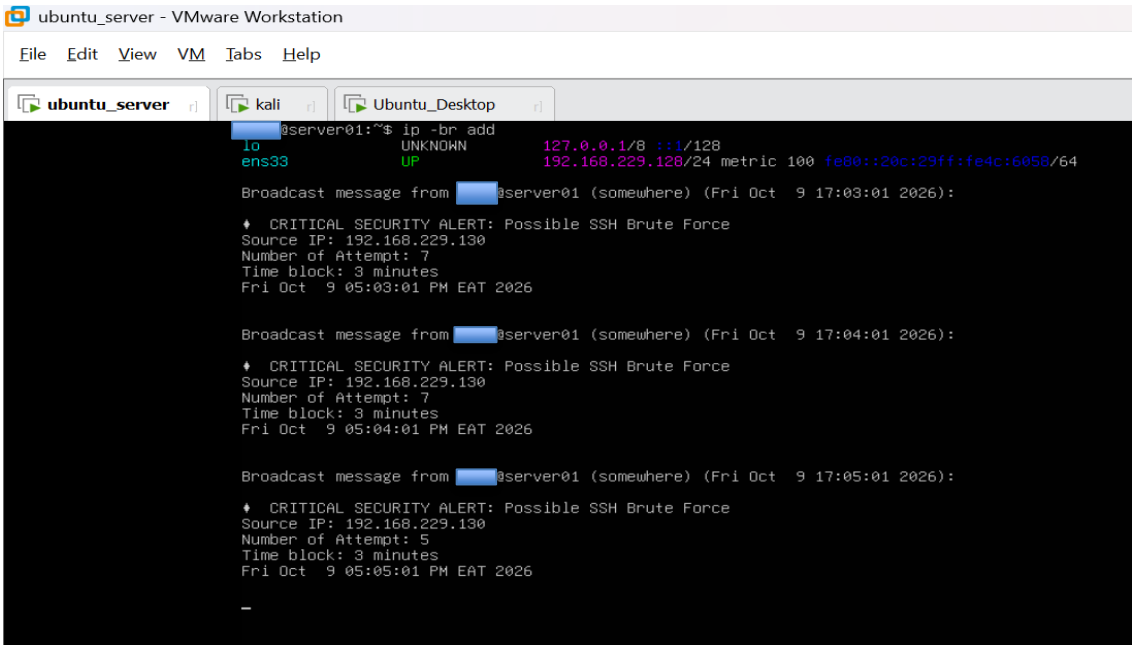


Figure 3. Second SSH test launched from Ubuntu Desktop.

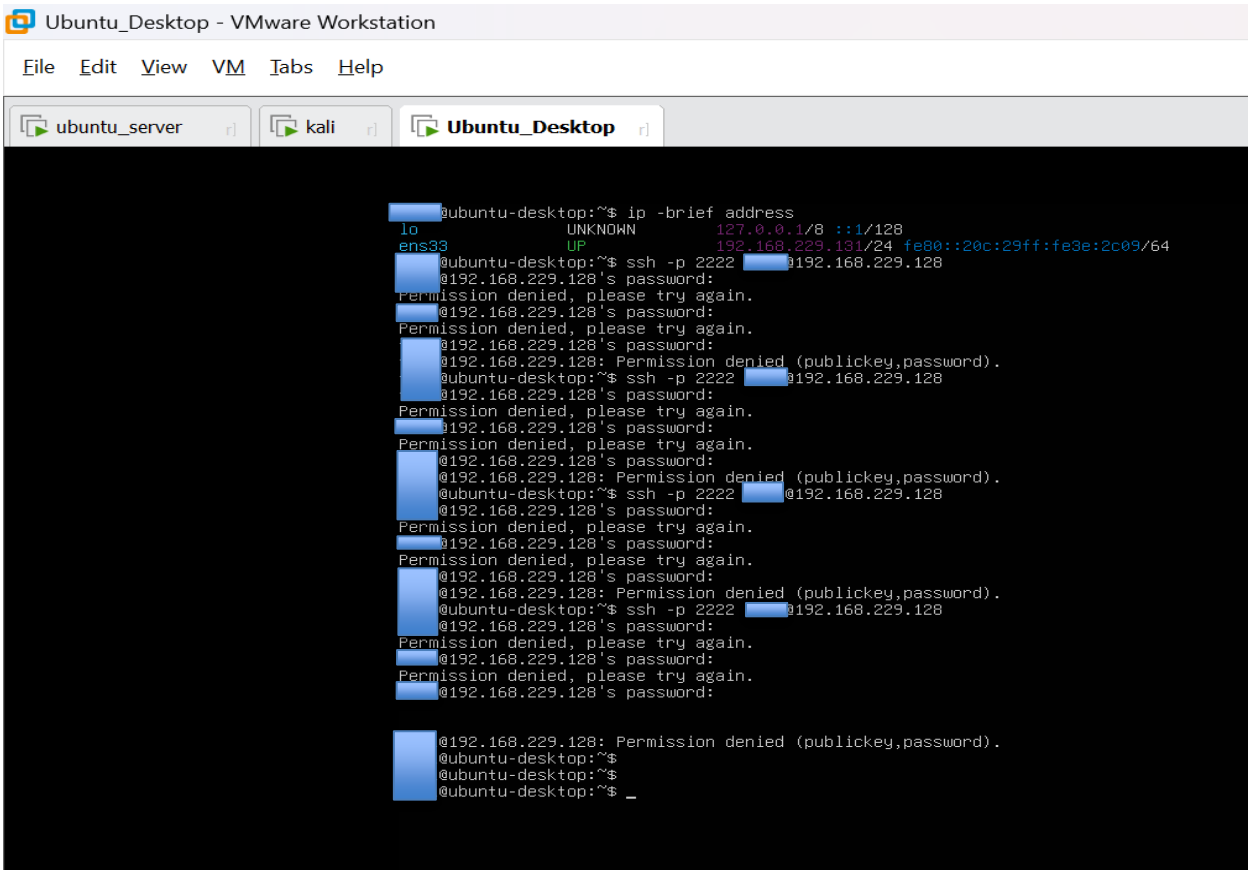



Figure 4. Ubuntu Server records the second test activity.

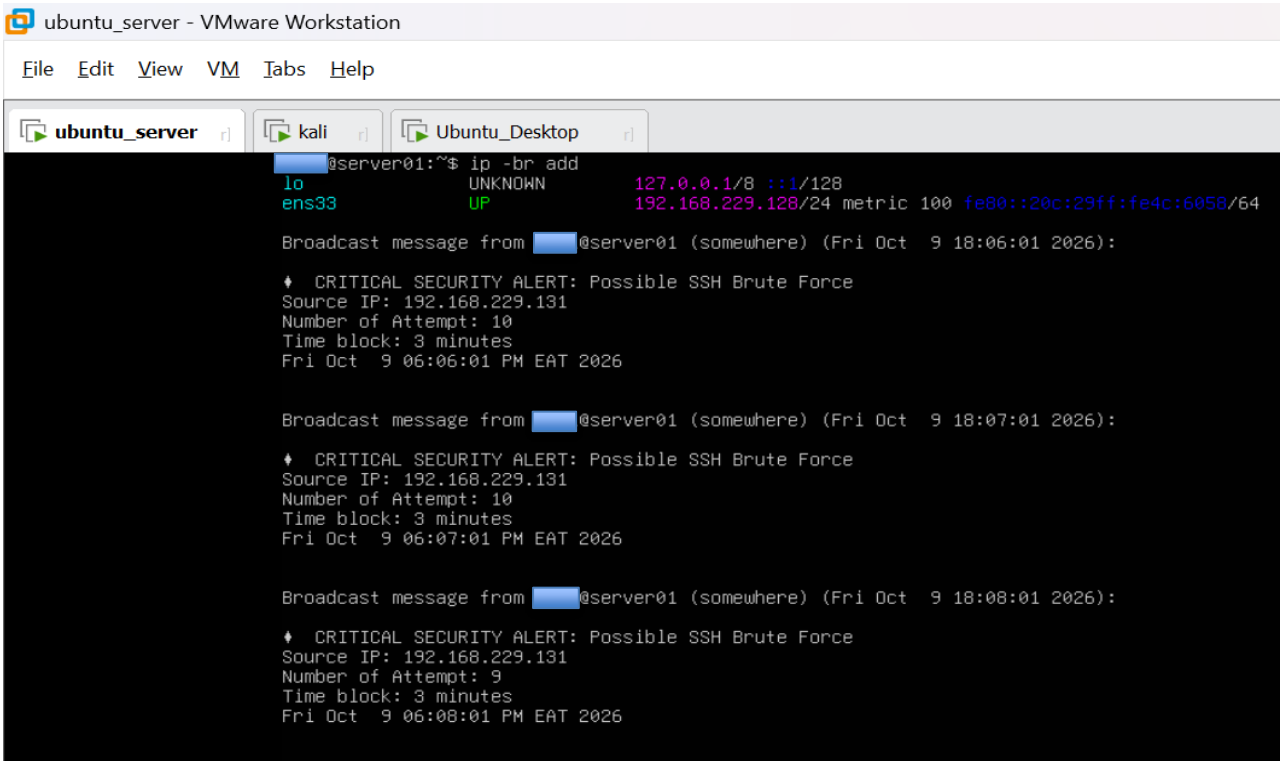


Figure 5. Detector script ‘alert.sh’ shown on Ubuntu Server.

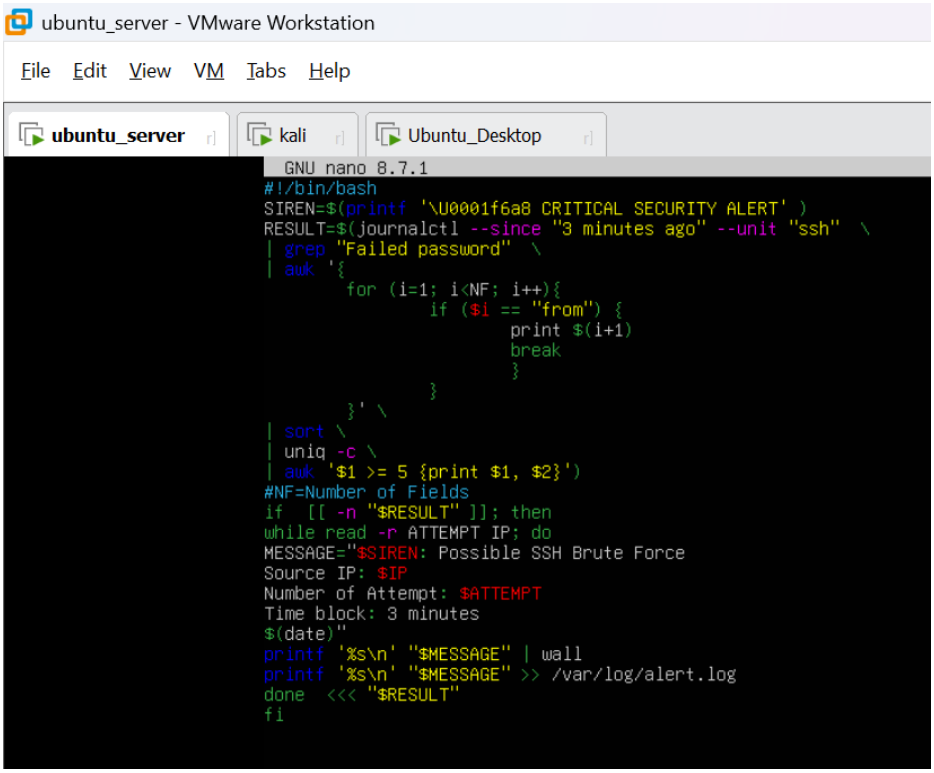


Figure 6. SSH configuration showing the Port 2222 directive.

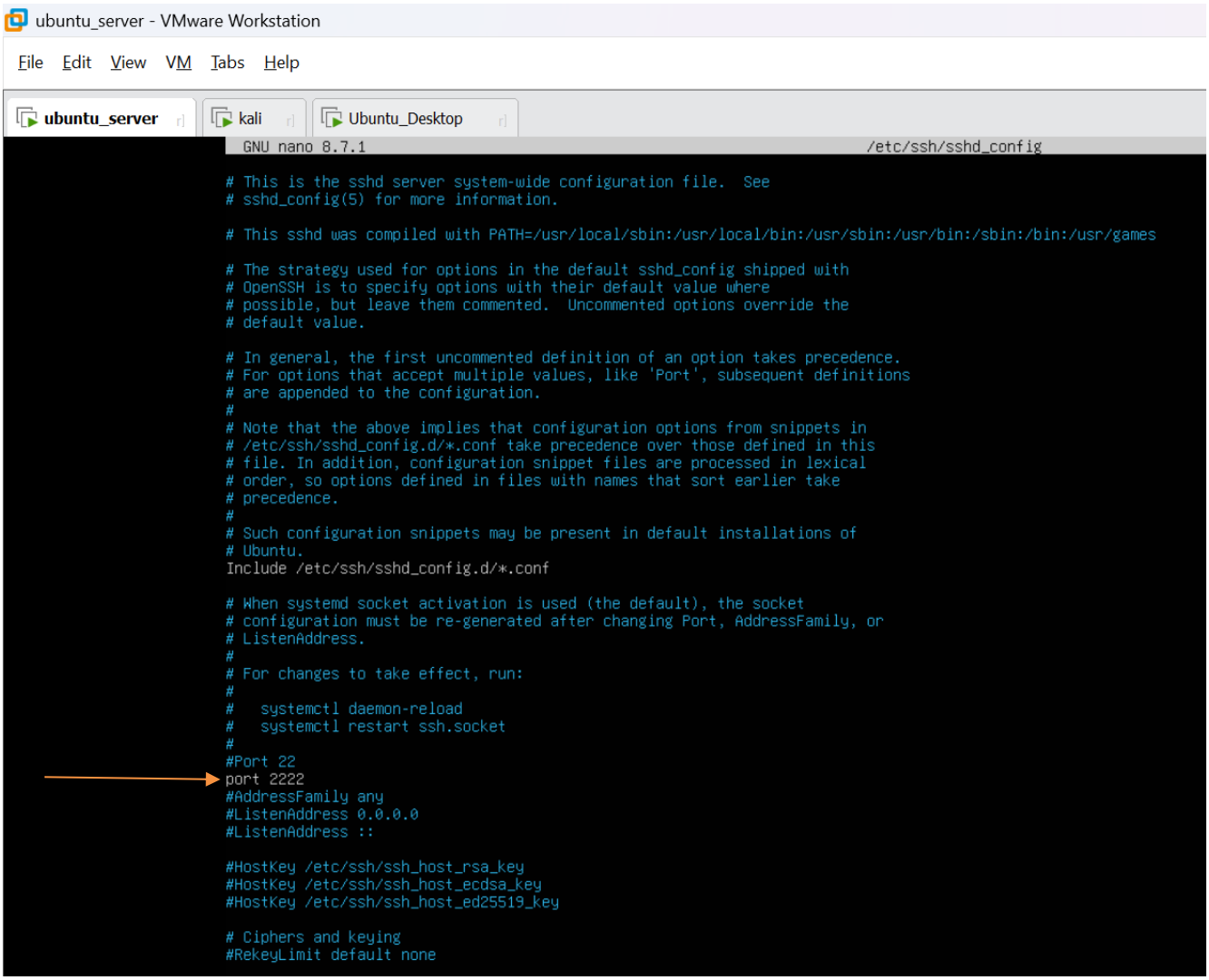


Figure 7. UFW status before the rule-order change, showing the source-specific REJECT rule after a broader ALLOW rule.

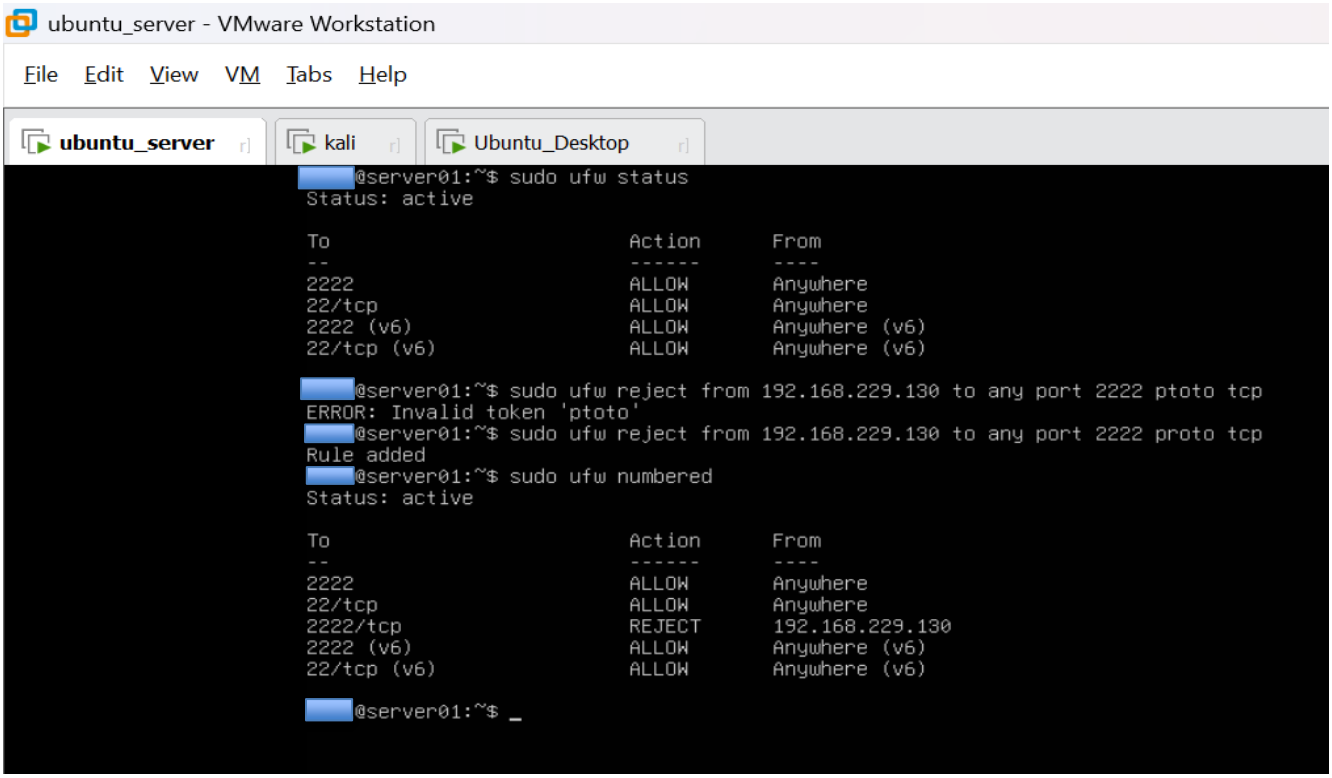



Figure 8. Kali connection attempt during troubleshooting, before the final rule-order retest.

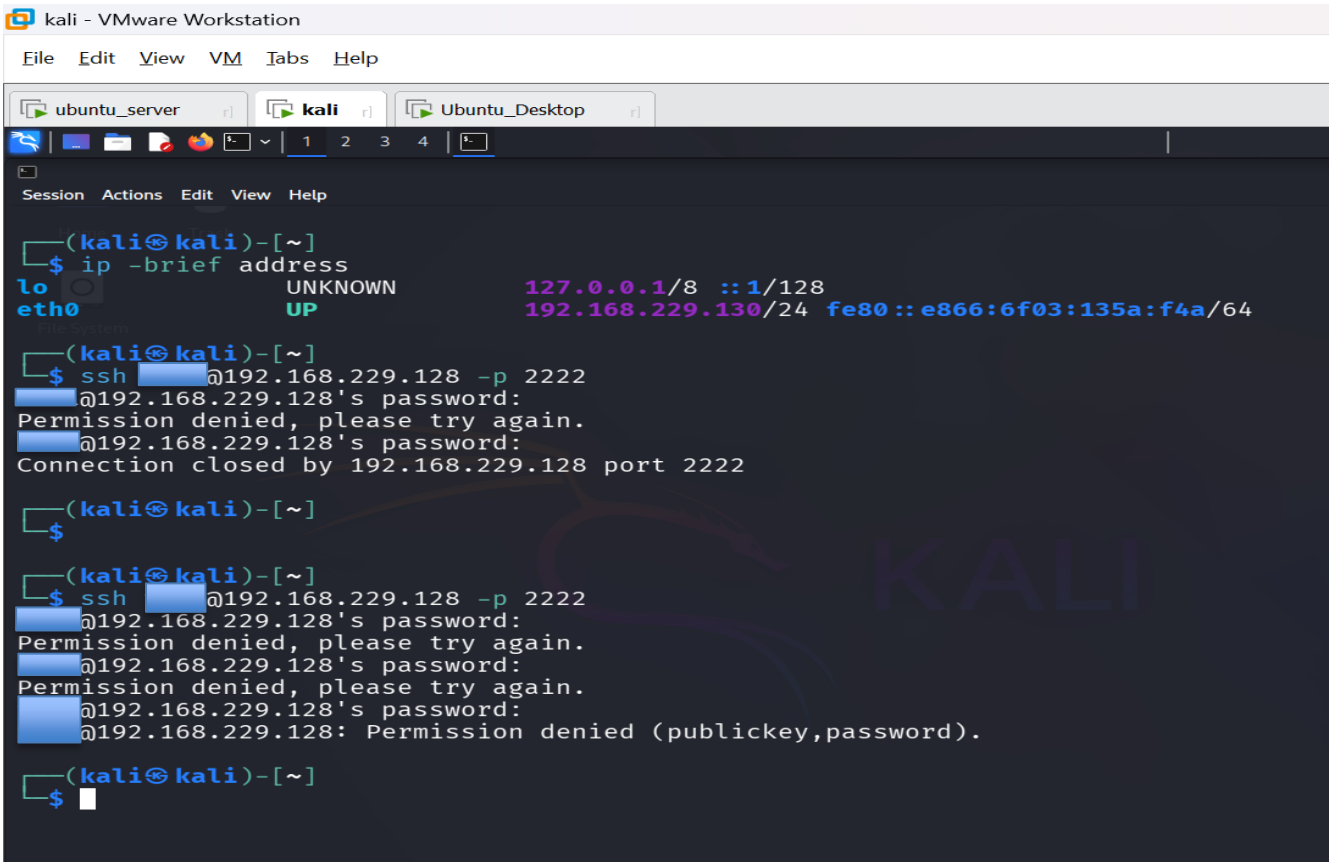


Figure 9. Ubuntu Server captures SSH traffic with tcpdump.

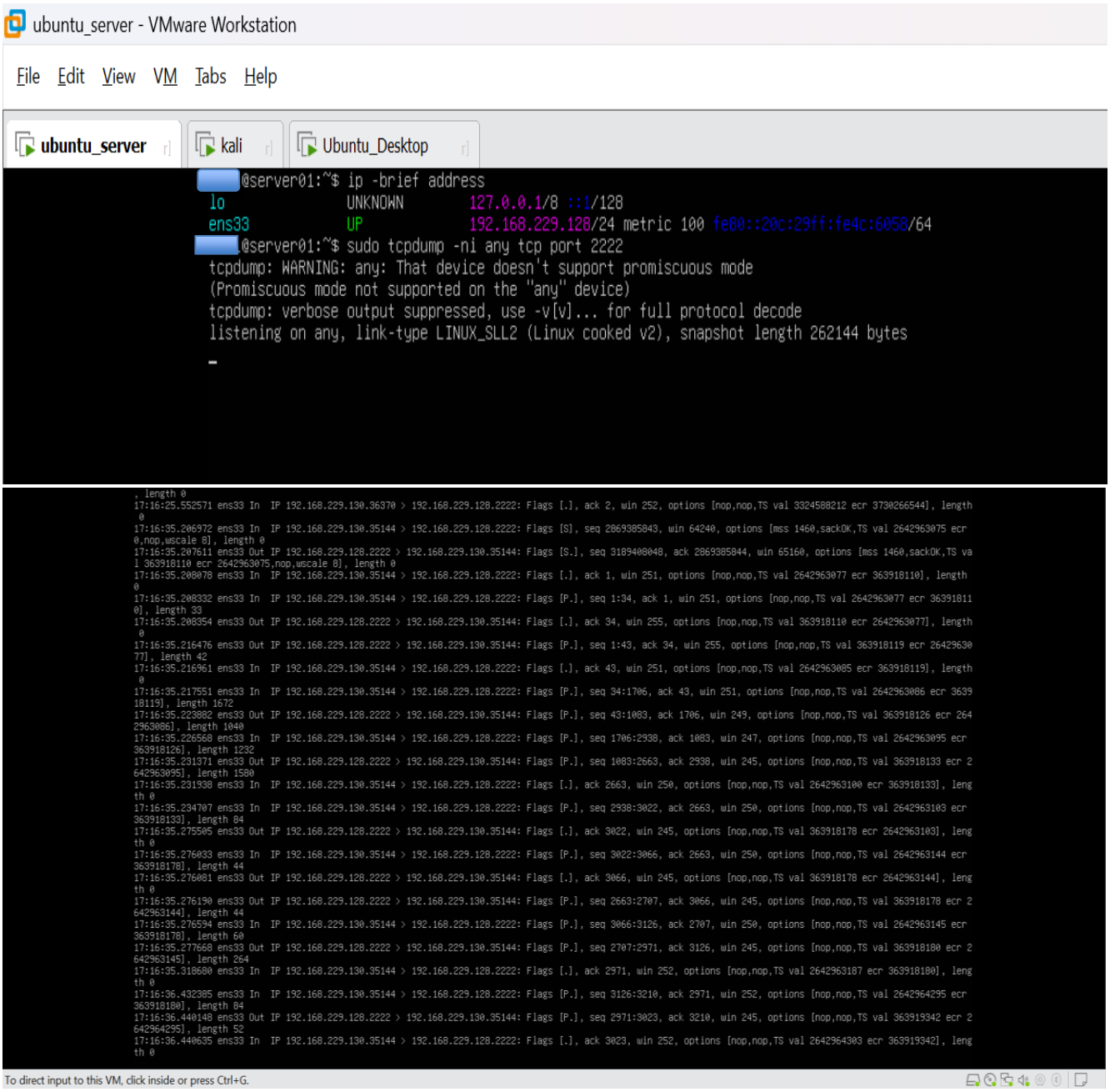


Figure 10. A new SSH connection attempt from Kali during troubleshooting.

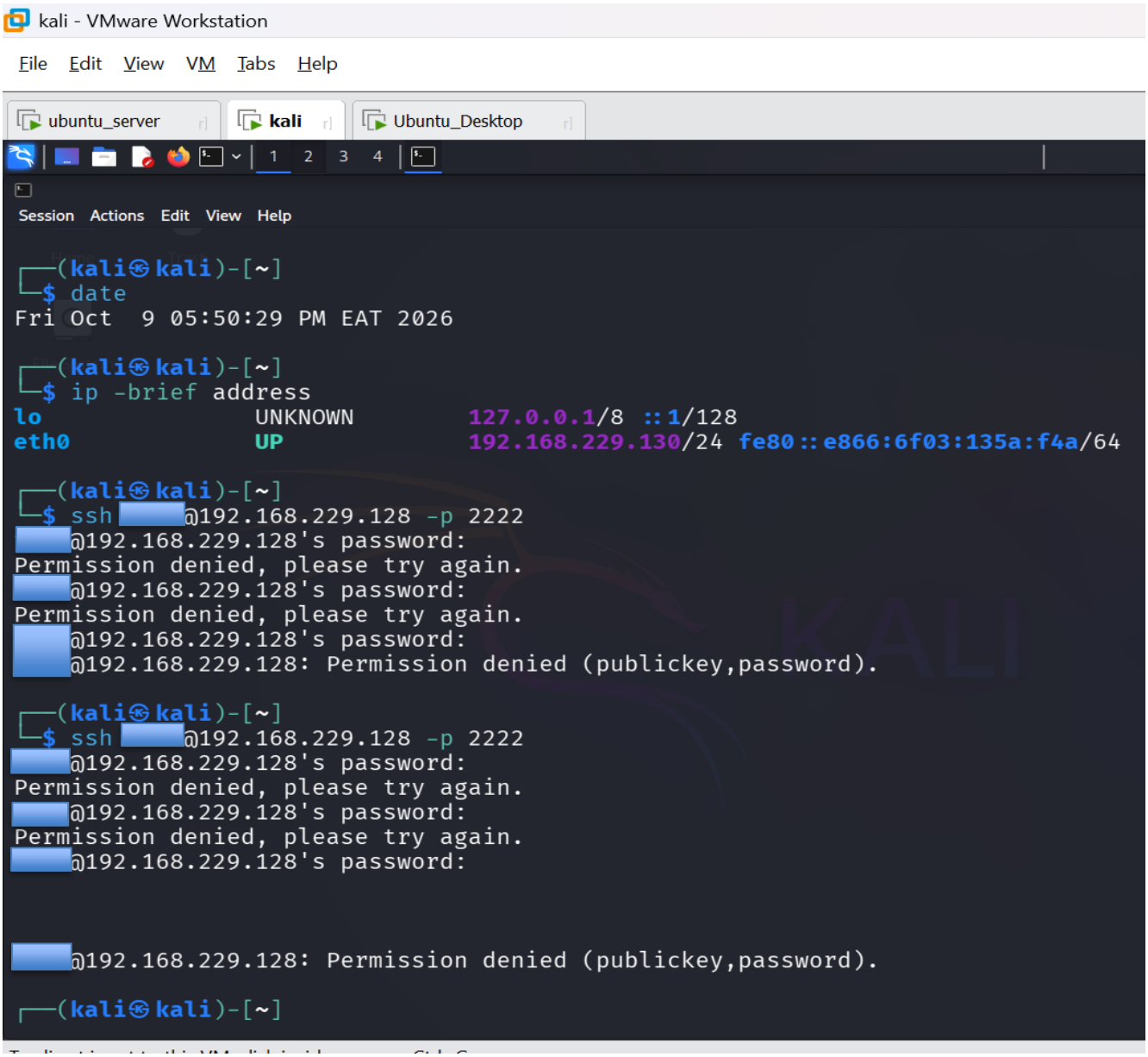


Figure 11. Packet capture ‘tcpdump’ showing source address 192.168.229.130.

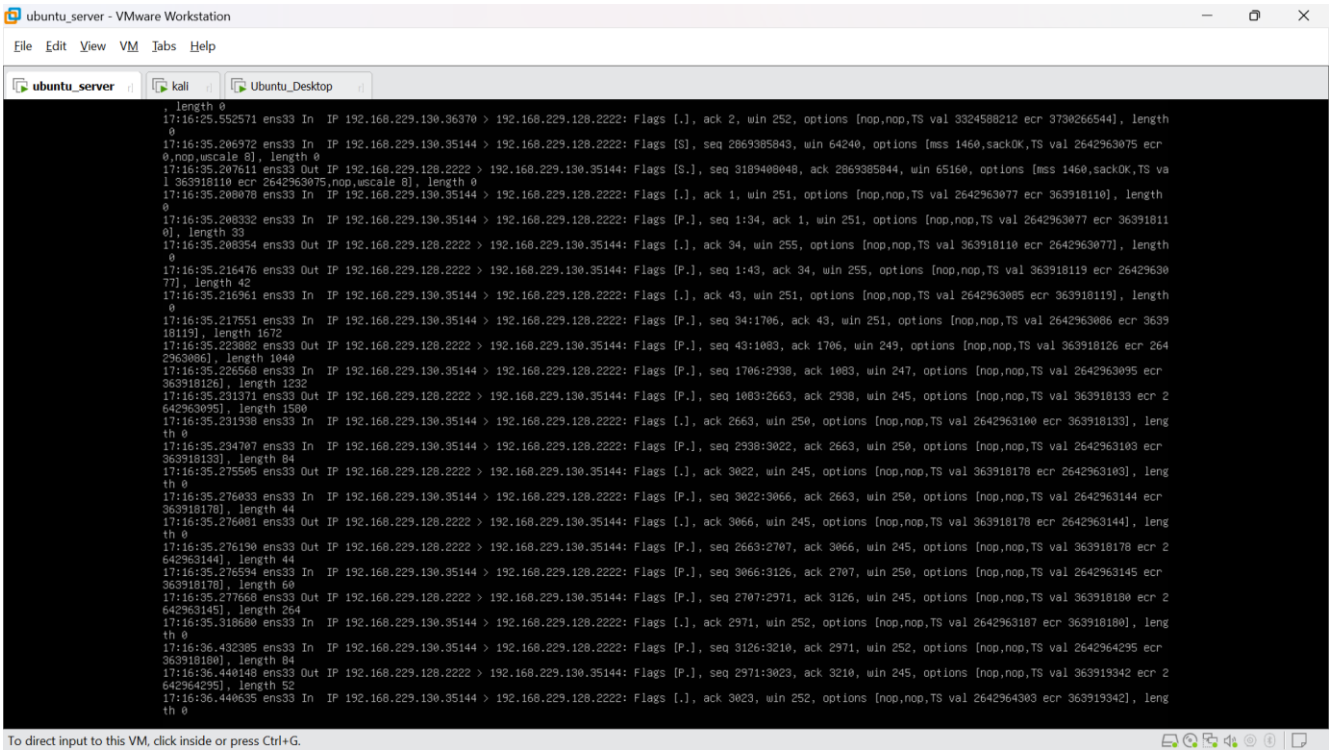


Figure 12. Alert broadcast displayed on Ubuntu Server.

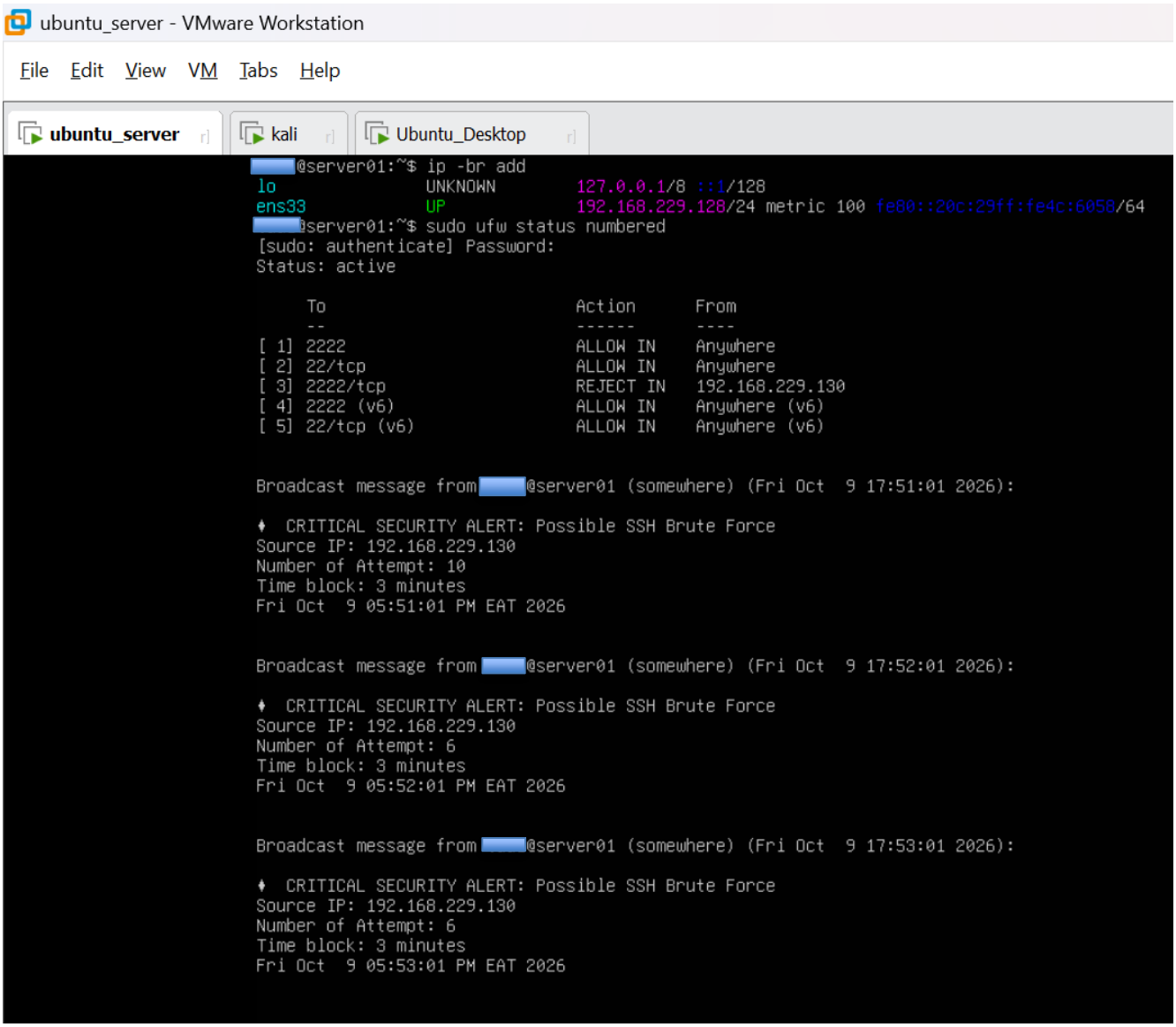


Figure 13. UFW rules after placing the source-specific reject rule ahead of the broad allow rule.

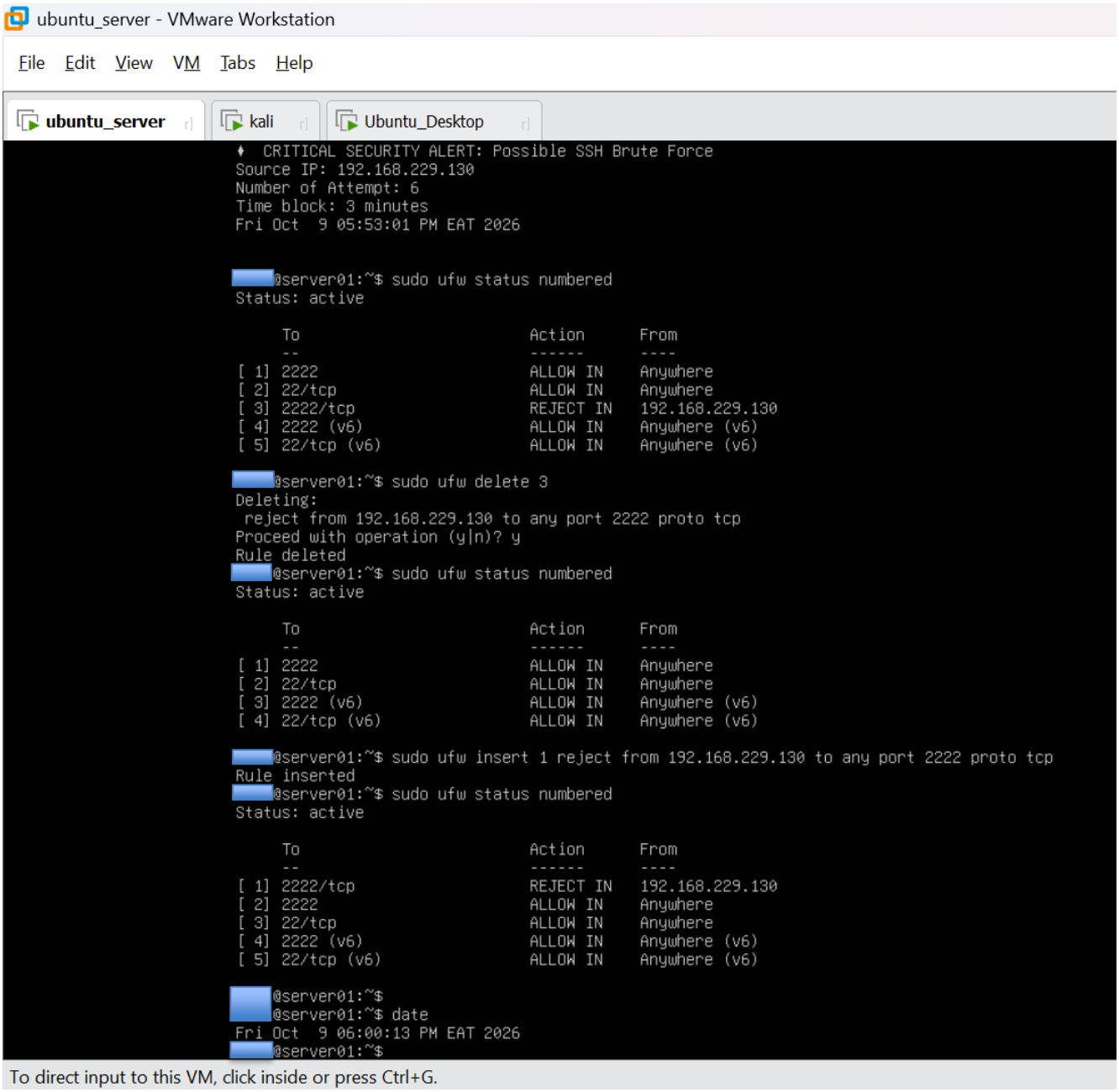


Figure 14. Kali connection retest after the source-specific reject rule was inserted at position 1; the screenshot documents the observed client result.

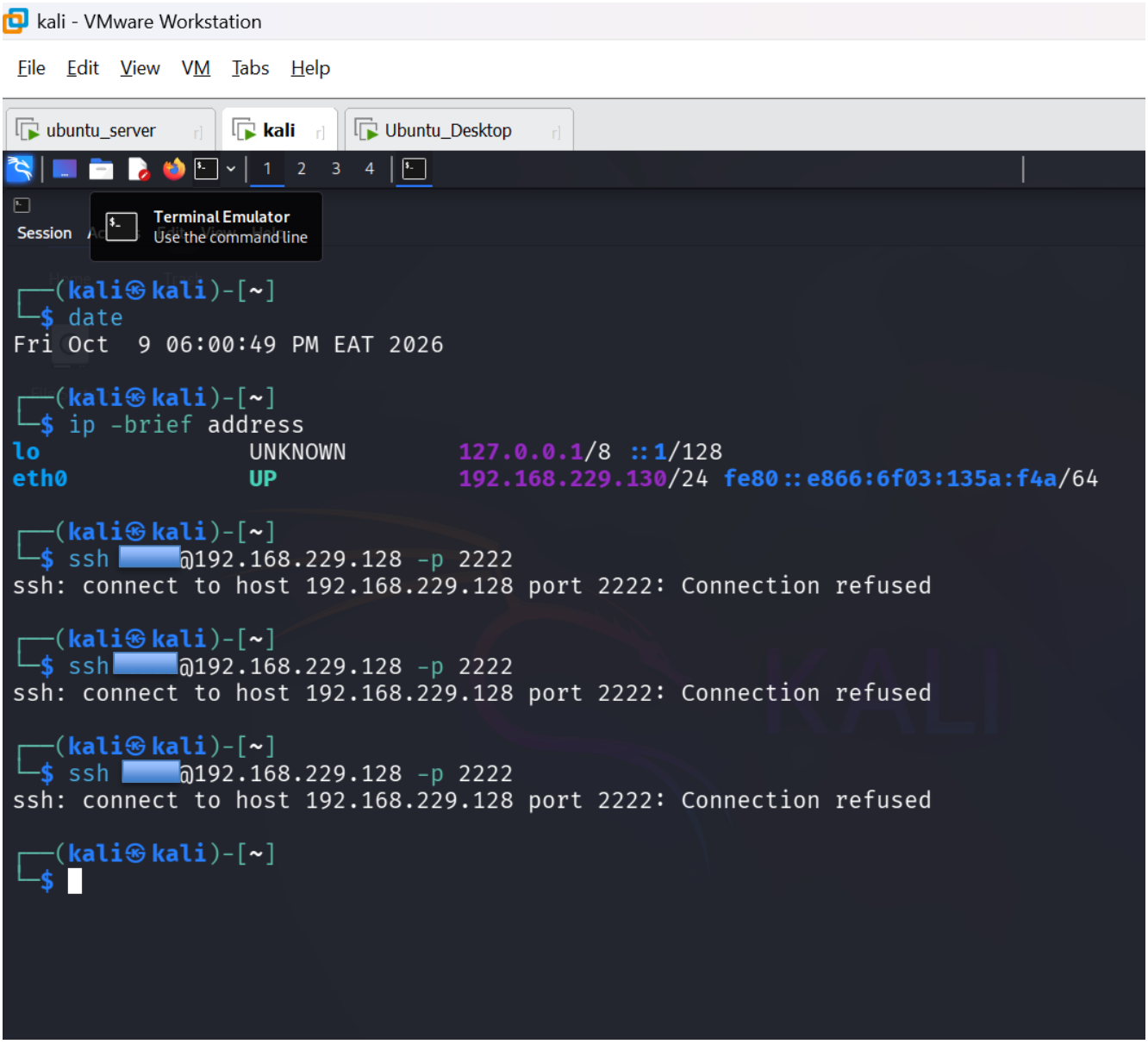


**Figure 15. Detection of SSH activity from multiple source IP addresses. The Ubuntu Server terminal shows separate alerts for 192.168.229.130 and 192.168.229.131, with six failed-password events attributed to each source within the configured three-minute lookback window.**

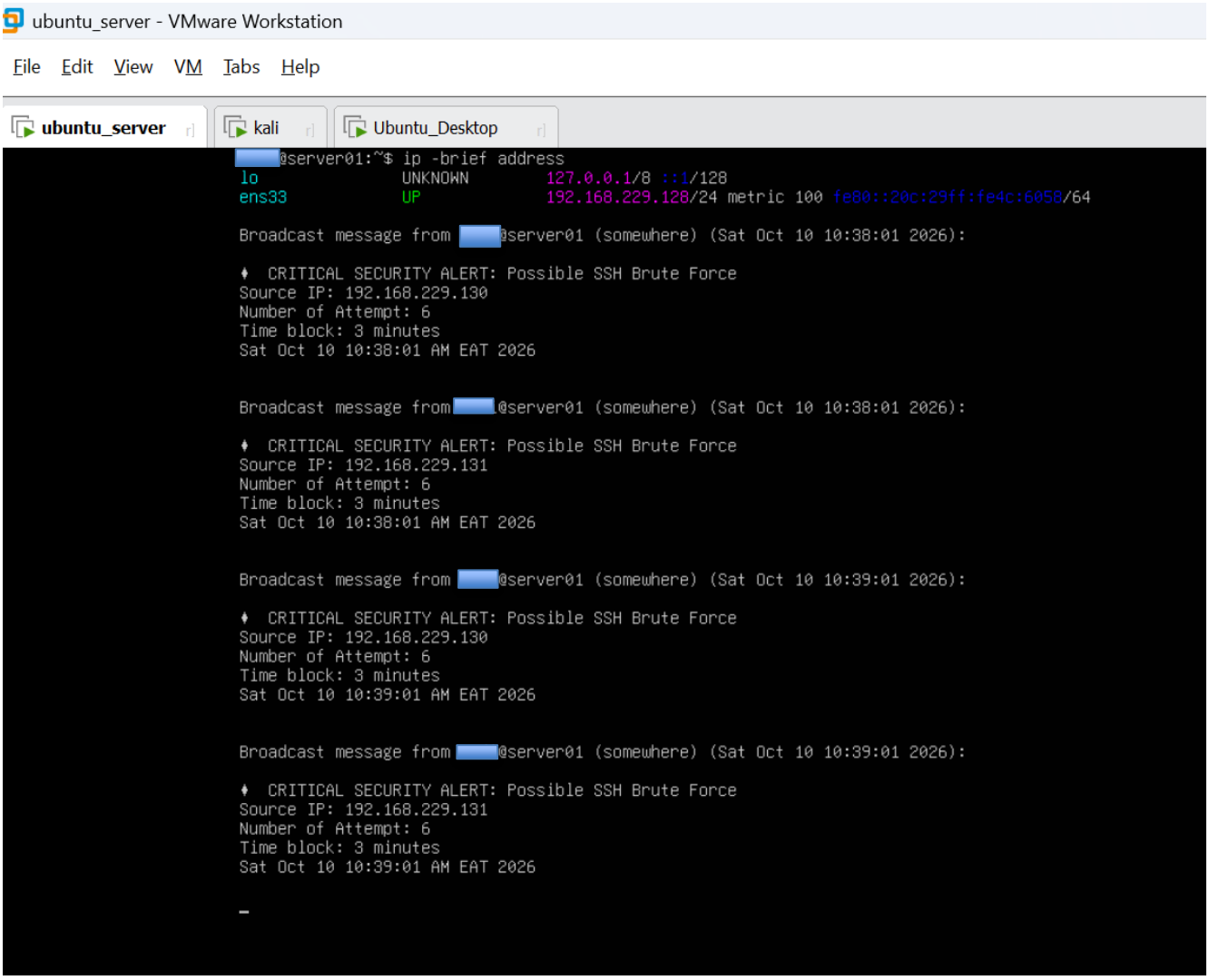
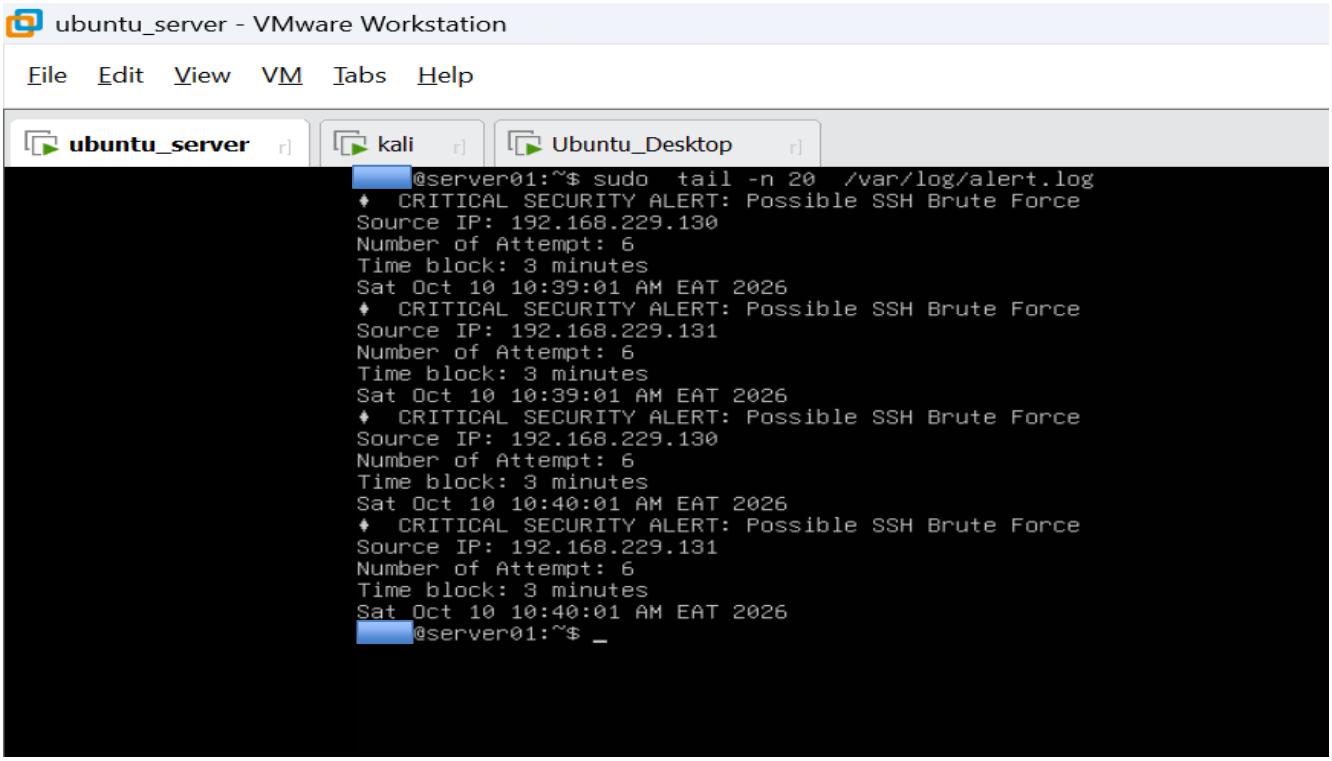


**Figure 16. Persistent SSH alert log. The /var/log/alert.log file records alerts generated by the detection script, including source IP address, failed-authentication count, configured lookback window, and alert timestamp.**




## 10. Limitations

- Detection depends on the SSH journal messages available on the target and on the fields and message formats parsed by the script.
- Cron-based polling introduces detection delay; the detector is not event-driven.
- The one-minute schedule and rolling three-minute lookback window can cause the same events to be counted in successive runs. Repeated alerts were observed during testing; alert deduplication is not implemented.
- A threshold of failed-authentication events is a signal for investigation, not proof of malicious intent.
- The firewall response described here is applied manually. The detector does not automatically block source IP addresses.
- Testing demonstrated separate alerts for two source IP addresses, but did not comprehensively validate simultaneous attack traffic, IPv6 connectivity and filtering, or all threshold boundary conditions. These scenarios remain untested and should be evaluated before stronger performance claims are made.

## 11. Lessons Learned

- Base conclusions on observable evidence: source IP addresses were checked against SSH logs and packet captures.
- Distinguish network access control from authentication: UFW filtering and SSH public-key authentication operate at different stages.
- Inspect the complete firewall ruleset and rule ordering rather than checking only whether a rule exists.
- Distinguish service state, socket activation, listening ports, and firewall policy.

- A package-manager lock usually indicates another package operation is in progress; identify the owning process and wait if it is progressing normally.
- Document each test, the observed output, the conclusion supported by that output, and any remaining uncertainty.

## 12. Future Improvements

- Validate the detector with controlled tests below, at, and above the threshold—for example, four, five, and six failed attempts—and record the expected and observed result for each case.
- Improve log parsing and test it against representative SSH log formats, usernames, IPv4/IPv6 addresses where applicable, and non-failure messages.
- Create a repeatable test matrix covering below-threshold, at-threshold, above-threshold, multiple-source, events outside the time window, and repeated runs over overlapping windows. Record pass/fail outcomes and retain sanitized evidence.
- After validating the basic detector, evaluate event-driven log monitoring or a centralized platform such as Wazuh if richer alerting, correlation, and investigation features are required.
- Document an incident-response procedure. If automated blocking is explored, use safeguards, expiry or unblock mechanisms, logging, and controlled tests to reduce the risk of locking out legitimate users.
- Include a concise README, configuration details, operating-system versions, test cases, and sanitized evidence so another person can reproduce the experiment.

## 13. Conclusion

- This lab demonstrates a practical workflow for SSH security monitoring: collecting authentication logs, filtering failed-password messages, grouping events by source IP, generating terminal alerts, and investigating firewall behavior. In the documented multiple-source test, the detector generated separate alerts for two source IP addresses, each with six failed-password events within the configured three-minute lookback window. During firewall troubleshooting, inserting a source-specific reject rule ahead of a broader allow rule was followed by a connection refusal in the subsequent test. The project also identifies limitations involving repeated alerts, threshold validation, log parsing, and IPv6 testing.
- Overall, this project demonstrates a practical, hands-on approach to SSH security monitoring, including log analysis, Bash-based detection, alert generation, and firewall troubleshooting. Further testing and documentation are needed to validate the detector across additional scenarios and assess its suitability for operational deployment.